

Notebook code reference

All notebook code is written without custom `def` or `lambda`. Each model has explicit training and metric statements. Cell positions include Markdown cells. Numbers before `|` are study annotations, not executable Python. Physical JSON source-block lines may move after reruns.

Notebook cell 2

Original file: `new.ipynb`, JSON source block lines 35?52.

Import the analysis/preprocessing libraries; warning suppression hides warnings rather than fixing them.

```
01 |                                     # Cell 2 - Imports and setup
02 | import warnings
03 | warnings.filterwarnings('ignore')
04 |
05 | import numpy as np
06 | import pandas as pd
07 | import matplotlib.pyplot as plt
08 | import seaborn as sns
09 | from sklearn.model_selection import train_test_split
10 | from sklearn.compose import ColumnTransformer
11 | from sklearn.pipeline import Pipeline
12 | from sklearn.impute import SimpleImputer
13 | from sklearn.preprocessing import OneHotEncoder, StandardScaler
14 |
15 | print('Libraries imported successfully.')
16 | print('Notebook ready for hotel booking analysis.')
```

Notebook cell 3

Original file: `new.ipynb`, JSON source block lines 70?78.

Read CSV into `df`; print original shape and row/column counts.

```
01 | # Cell 3 - Load the dataset
02 | file_path = 'hotel_bookings.csv'
03 | df = pd.read_csv(file_path)
04 |
05 | print('Dataset shape:', df.shape)
06 | print('Number of rows:', df.shape[0])
07 | print('Number of columns:', df.shape[1])
```

Notebook cell 4

Original file: `new.ipynb`, JSON source block lines 97?100.

Preview five bookings and connect field values to the scenario.

```
01 | # Cell 4 - Preview the dataset
02 | df.head().to_string(index=False)
```

Notebook cell 5

Original file: new.ipynb, JSON source block lines 150?153.

List original columns, including target and outcome-related fields.

```
01 | # Cell 5 - Show column names
02 | list(df.columns)
```

Notebook cell 6

Original file: new.ipynb, JSON source block lines 204?207.

Inspect data types before choosing different transformations.

```
01 | # Cell 6 - Check data types
02 | df.dtypes
```

Notebook cell 7

Original file: new.ipynb, JSON source block lines 261?264.

Display types, non-null counts, and memory usage.

```
01 | # Cell 7 - Basic information about the data
02 | df.info()
```

Notebook cell 8

Original file: new.ipynb, JSON source block lines 287?291.

Count and rank missing fields: company, agent, country, children.

```
01 | # Cell 8 - Check missing values
02 | missing = df.isna().sum()
03 | missing[missing > 0].sort_values(ascending=False)
```

Notebook cell 9

Original file: new.ipynb, JSON source block lines 310?313.

Count 31,994 exact repeated rows.

```
01 | # Cell 9 - Check duplicate records
02 | df.duplicated().sum()
```

Notebook cell 10

Original file: new.ipynb, JSON source block lines 335?338.

Count row labels: 75,166 zeros and 44,224 ones.

```
01 | # Cell 10 - Inspect target variable distribution
02 | df['is_canceled'].value_counts()
```

Notebook cell 11

Original file: new.ipynb, JSON source block lines 360?363.

Calculate raw target proportions: 62.96% / 37.04%.

```
01 | # Cell 11 - Check target proportion
02 | df['is_canceled'].value_counts(normalize=True)
```

Notebook cell 13

Original file: new.ipynb, JSON source block lines 399?408.

Plot label counts; explain moderate raw class imbalance.

```
01 | # Cell 12 - Visualization 1: cancellation target balance
02 | plt.figure(figsize=(8, 5))
03 | sns.countplot(data=df, x='is_canceled', palette='Set2')
04 | plt.title('Cancellation Distribution')
05 | plt.xlabel('Booking Status')
06 | plt.ylabel('Count')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 14

Original file: new.ipynb, JSON source block lines 437?449.

Mean binary target by hotel gives City 41.73% and Resort 27.76% raw cancellation rates.

```
01 | # Cell 13 - Group cancellation rate by hotel type
02 | hotel_rate = df.groupby('hotel')['is_canceled'].mean().sort_values(ascending=False)
03 | print(hotel_rate)
04 |
05 | plt.figure(figsize=(8, 5))
06 | sns.barplot(data=df, x='hotel', y='is_canceled', estimator='mean', palette='Set2')
07 | plt.title('Cancellation Rate by Hotel Type')
08 | plt.xlabel('Hotel')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 15

Original file: new.ipynb, JSON source block lines 488?501.

Reindex months to calendar order and plot mean target by month.

```
01 | # Cell 14 - Month-wise cancellation patterns
02 | month_order =
    ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
03 | monthly_rate = df.groupby('arrival_date_month')['is_canceled'].mean().reindex(month_order)
04 | print(monthly_rate)
05 |
06 | plt.figure(figsize=(12, 5))
07 | sns.barplot(data=df, x='arrival_date_month', y='is_canceled', estimator='mean', order=month_order,
    palette='viridis')
08 | plt.title('Cancellation Rate by Arrival Month')
09 | plt.xticks(rotation=45)
10 | plt.ylabel('Cancellation Rate')
11 | plt.tight_layout()
12 | plt.show()
```

Notebook cell 16

Original file: new.ipynb, JSON source block lines 532?544.

Compare customer-type cancellation proportions.

```
01 | # Cell 15 - Customer type analysis
02 | customer_rate = df.groupby('customer_type')['is_canceled'].mean().sort_values(ascending=False)
03 | print(customer_rate)
04 |
05 | plt.figure(figsize=(10, 5))
06 | sns.barplot(data=df, x='customer_type', y='is_canceled', estimator='mean', palette='pastel')
07 | plt.title('Cancellation Rate by Customer Type')
08 | plt.xlabel('Customer Type')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 17

Original file: new.ipynb, JSON source block lines 579?591.

Compare market segments; Undefined has only two rows.

```
01 | # Cell 16 - Market segment analysis
02 | market_rate = df.groupby('market_segment')['is_canceled'].mean().sort_values(ascending=False)
03 | print(market_rate)
04 |
05 | plt.figure(figsize=(10, 5))
06 | sns.barplot(data=df, x='market_segment', y='is_canceled', estimator='mean', palette='magma')
07 | plt.title('Cancellation Rate by Market Segment')
08 | plt.xlabel('Market Segment')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 18

Original file: new.ipynb, JSON source block lines 621?633.

Compare deposit types; Non Refund association is not proof of cause.

```
01 | # Cell 17 - Deposit type analysis
02 | deposit_rate = df.groupby('deposit_type')['is_canceled'].mean().sort_values(ascending=False)
03 | print(deposit_rate)
04 |
05 | plt.figure(figsize=(9, 5))
06 | sns.barplot(data=df, x='deposit_type', y='is_canceled', estimator='mean', palette='husl')
07 | plt.title('Cancellation Rate by Deposit Type')
08 | plt.xlabel('Deposit Type')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 19

Original file: new.ipynb, JSON source block lines 665?677.

Compare distribution-channel cancellation rates; check category sample sizes.

```
01 | # Cell 18 - Distribution channel analysis
02 | channel_rate = df.groupby('distribution_channel')['is_canceled'].mean().sort_values(ascending=False)
03 | print(channel_rate)
04 |
05 | plt.figure(figsize=(9, 5))
06 | sns.barplot(data=df, x='distribution_channel', y='is_canceled', estimator='mean', palette='coolwarm')
07 | plt.title('Cancellation Rate by Distribution Channel')
08 | plt.xlabel('Channel')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 20

Original file: new.ipynb, JSON source block lines 714?726.

Compare reserved-room codes; P has only twelve raw rows.

```
01 | # Cell 19 - Room type analysis
02 | room_rate = df.groupby('reserved_room_type')['is_canceled'].mean().sort_values(ascending=False)
03 | print(room_rate)
04 |
05 | plt.figure(figsize=(10, 5))
06 | sns.barplot(data=df, x='reserved_room_type', y='is_canceled', estimator='mean', palette='viridis')
07 | plt.title('Cancellation Rate by Reserved Room Type')
08 | plt.xlabel('Room Type')
09 | plt.ylabel('Cancellation Rate')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 21

Original file: new.ipynb, JSON source block lines 745?754.

Histogram/KDE of lead_time describes booking-to-arrival days.

```
01 | # Cell 20 - Lead time distribution and business meaning
02 | plt.figure(figsize=(10, 5))
03 | sns.histplot(df['lead_time'], bins=30, kde=True, color='royalblue')
04 | plt.title('Distribution of Lead Time')
05 | plt.xlabel('Lead Time (days)')
06 | plt.ylabel('Frequency')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 22

Original file: new.ipynb, JSON source block lines 773?782.

Histogram/KDE of ADR describes price distribution and extreme values.

```
01 | # Cell 21 - ADR distribution
02 | plt.figure(figsize=(10, 5))
03 | sns.histplot(df['adr'], bins=30, kde=True, color='tomato')
04 | plt.title('Distribution of ADR')
05 | plt.xlabel('ADR')
06 | plt.ylabel('Frequency')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 23

Original file: new.ipynb, JSON source block lines 801?810.

Box plot compares lead-time medians/spread by outcome.

```
01 | # Cell 22 - Lead time comparison by cancellation status
02 | plt.figure(figsize=(10, 5))
03 | sns.boxplot(data=df, x='is_canceled', y='lead_time', palette='Set2')
04 | plt.title('Lead Time by Cancellation Status')
05 | plt.xlabel('Booked Status')
06 | plt.ylabel('Lead Time (days)')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 24

Original file: new.ipynb, JSON source block lines 829?838.

Box plot compares ADR by outcome; no outlier treatment is applied.

```
01 | # Cell 23 - ADR comparison by cancellation status
02 | plt.figure(figsize=(10, 5))
03 | sns.boxplot(data=df, x='is_canceled', y='adr', palette='Set2')
```

```
04 | plt.title('ADR by Cancellation Status')
05 | plt.xlabel('Booked Status')
06 | plt.ylabel('ADR')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 25

Original file: new.ipynb, JSON source block lines 859?864.

Compute total_nights as weekend nights plus week nights for EDA.

```
01 | # Cell 24 - Compute stay length feature
02 | # This is a useful derived variable for later modeling.
03 | df['total_nights'] = df['stays_in_weekend_nights'] + df['stays_in_week_nights']
04 | print(df[['stays_in_weekend_nights', 'stays_in_week_nights', 'total_nights']].head())
```

Notebook cell 26

Original file: new.ipynb, JSON source block lines 883?892.

Box plot compares total stay length by outcome.

```
01 | # Cell 25 - Stay length by cancellation status
02 | plt.figure(figsize=(10, 5))
03 | sns.boxplot(data=df, x='is_canceled', y='total_nights', palette='Set2')
04 | plt.title('Stay Length by Cancellation Status')
05 | plt.xlabel('Canceled?')
06 | plt.ylabel('Total Nights')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 27

Original file: new.ipynb, JSON source block lines 921?933.

Compare mean special requests: raw non-canceled 0.7141 vs canceled 0.3288.

```
01 | # Cell 26 - Special requests and cancellation relationship
02 | special_request_rate = df.groupby('is_canceled')['total_of_special_requests'].mean()
03 | print(special_request_rate)
04 |
05 | plt.figure(figsize=(10, 5))
06 | sns.barplot(data=df, x='is_canceled', y='total_of_special_requests', estimator='mean', palette='Set2')
07 | plt.title('Average Special Requests by Cancellation Status')
08 | plt.xlabel('Canceled?')
09 | plt.ylabel('Special Requests')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 28

Original file: new.ipynb, JSON source block lines 970?982.

Count top ten countries; counts measure concentration rather than cancellation rates.

```
01 | # Cell 27 - Country concentration
02 | country_counts = df['country'].value_counts().head(10)
03 | print(country_counts)
04 |
05 | plt.figure(figsize=(12, 5))
06 | country_counts.plot(kind='bar', color='steelblue')
07 | plt.title('Top 10 Countries by Booking Count')
08 | plt.xlabel('Country')
09 | plt.ylabel('Count')
10 | plt.tight_layout()
11 | plt.show()
```

Notebook cell 29

Original file: new.ipynb, JSON source block lines 1002?1007.

Report agent/company missingness: 13.69% and 94.31%.

```
01 | # Cell 28 - Missing values in agent and company columns
02 | print('Agent missing %:', round(df['agent'].isna().mean() * 100, 2))
03 | print('Company missing %:', round(df['company'].isna().mean() * 100, 2))
04 | print(df[['agent', 'company']].isna().sum())
```

Notebook cell 30

Original file: new.ipynb, JSON source block lines 1026?1035.

Calculate numeric correlations and plot heatmap; association is not causality.

```
01 | # Cell 29 - Correlation heatmap for numerical features
02 | numeric_corr = df.select_dtypes(include=['number']).corr()
03 |
04 | plt.figure(figsize=(12, 10))
05 | sns.heatmap(numeric_corr, cmap='coolwarm', annot=False)
06 | plt.title('Numerical Feature Correlation Heatmap')
07 | plt.tight_layout()
08 | plt.show()
```

Notebook cell 31

Original file: new.ipynb, JSON source block lines 1051?1055.

Identify numeric fields; IDs are reconsidered as categories in Stage 6.

```
01 | # Cell 30 - Summary of numeric columns
02 | numeric_cols = df.select_dtypes(include=['number']).columns.tolist()
03 | print(numeric_cols)
```

Notebook cell 32

Original file: new.ipynb, JSON source block lines 1162?1165.

Inspect count/mean/std/quartiles/extrema; ADR ranges from -6.38 to 5400.

```
01 | # Cell 31 - Summary statistics for key numeric variables
02 | df[['lead_time', 'adr', 'total_nights', 'days_in_waiting_list']].describe().T
```

Notebook cell 33

Original file: new.ipynb, JSON source block lines 1184?1190.

Count distinct hotels/countries/segments/customer types: 2/177/8/4.

```
01 | # Cell 32 - Check unique counts for some categorical variables
02 | print('Unique hotels:', df['hotel'].nunique())
03 | print('Unique countries:', df['country'].nunique())
04 | print('Unique market segments:', df['market_segment'].nunique())
05 | print('Unique customer types:', df['customer_type'].nunique())
```

Notebook cell 34

Original file: new.ipynb, JSON source block lines 1207?1213.

Flag reservation_status and reservation_status_date as leakage fields.

```
01 | # Cell 33 - Leakage check
02 | # These fields are outcome-related and must not be used as input features.
03 | for col in ['reservation_status', 'reservation_status_date']:
04 |     if col in df.columns:
05 |         print(col, 'exists and should be excluded from prediction features.')
```

Notebook cell 35

Original file: new.ipynb, JSON source block lines 1230?1234.

Explain why final reservation status reveals the outcome.

```
01 | # Cell 34 - Result of the leakage review
02 | print('The variable reservation_status tells us if the booking was already canceled or completed.')
03 | print('Because it is a post-booking outcome, it would create data leakage if included in the model.')
```

Notebook cell 36

Original file: new.ipynb, JSON source block lines 1254?1261.

Summarize observed EDA associations; do not interpret wording as causal proof.

```
01 | # Cell 35 - Important EDA observation summary
02 | print('Key findings from EDA:')
03 | print('- Cancellation rate is not balanced across classes.')
04 | print('- Hotel type, month, customer type, and market segment affect cancellations.')
05 | print('- Lead time, deposit type, and ADR are strong behavioural signals.')
06 | print('- Some fields such as reservation_status must be removed to avoid leakage.')
```

Notebook cell 38

Original file: new.ipynb, JSON source block lines 1294?1298.

Copy df to retain the original EDA dataframe separately.

```
01 | # Cell 37 - Create a clean working copy
02 | cleaned_df = df.copy()
03 | print('Initial rows:', len(cleaned_df))
```

Notebook cell 39

Original file: new.ipynb, JSON source block lines 1314?1318.

Remove exact duplicates, leaving 87,396 rows and a changed label balance.

```
01 | # Cell 38 - Remove duplicate rows
02 | cleaned_df = cleaned_df.drop_duplicates()
03 | print('Rows after duplicate removal:', len(cleaned_df))
```

Notebook cell 40

Original file: new.ipynb, JSON source block lines 1334?1338.

Fill children using cleaned-data median; pre-split learning is a limitation.

```
01 | # Cell 39 - Fill missing values for children
02 | cleaned_df['children'] = cleaned_df['children'].fillna(cleaned_df['children'].median())
03 | print('Children missing after fill:', cleaned_df['children'].isna().sum())
```

Notebook cell 41

Original file: new.ipynb, JSON source block lines 1354?1364.

Fill missing categories with selected constants such as Unknown.

```
01 | # Cell 40 - Handle categorical missing values
02 | cleaned_df['country'] = cleaned_df['country'].fillna('Unknown')
03 | cleaned_df['meal'] = cleaned_df['meal'].fillna('SC')
04 | cleaned_df['market_segment'] = cleaned_df['market_segment'].fillna('Unknown')
05 | cleaned_df['distribution_channel'] = cleaned_df['distribution_channel'].fillna('Unknown')
06 | cleaned_df['customer_type'] = cleaned_df['customer_type'].fillna('Transient')
07 | cleaned_df['deposit_type'] = cleaned_df['deposit_type'].fillna('No Deposit')
```

```
08 |  
09 | print('Categorical missing values reduced.')
```

Notebook cell 42

Original file: new.ipynb, JSON source block lines 1381?1387.

Fill absent agent/company with zero and cast to integers.

```
01 | # Cell 41 - Handle agent and company missing values  
02 | cleaned_df['agent'] = cleaned_df['agent'].fillna(0).astype(int)  
03 | cleaned_df['company'] = cleaned_df['company'].fillna(0).astype(int)  
04 | print('Agent nulls:', cleaned_df['agent'].isna().sum())  
05 | print('Company nulls:', cleaned_df['company'].isna().sum())
```

Notebook cell 43

Original file: new.ipynb, JSON source block lines 1408?1416.

Recompute stay length and add strict >7, >0 weekend, and >60 indicators.

```
01 | # Cell 42 - Create engineered features  
02 | cleaned_df['total_nights'] = cleaned_df['stays_in_weekend_nights'] + cleaned_df['stays_in_week_nights']  
03 | cleaned_df['is_long_stay'] = (cleaned_df['total_nights'] > 7).astype(int)  
04 | cleaned_df['is_weekend_stay'] = (cleaned_df['stays_in_weekend_nights'] > 0).astype(int)  
05 | cleaned_df['is_high_lead_time'] = (cleaned_df['lead_time'] > 60).astype(int)  
06 |  
07 | print(cleaned_df[['total_nights', 'is_long_stay', 'is_weekend_stay', 'is_high_lead_time']].head())
```

Notebook cell 44

Original file: new.ipynb, JSON source block lines 1433?1439.

Remove both outcome fields; errors=ignore tolerates already absent columns.

```
01 | # Cell 43 - Drop leakage columns  
02 | leakage_cols = ['reservation_status', 'reservation_status_date']  
03 | cleaned_df = cleaned_df.drop(columns=leakage_cols, errors='ignore')  
04 | print('Leakage columns removed from the feature set.')  
05 | print('Remaining columns:', len(cleaned_df.columns))
```

Notebook cell 45

Original file: new.ipynb, JSON source block lines 1458?1461.

Check whether any missing values remain after cleaning.

```
01 | # Cell 44 - Check remaining missing values  
02 | cleaned_df.isna().sum()[cleaned_df.isna().sum() > 0]
```

Notebook cell 46

Original file: new.ipynb, JSON source block lines 1480?1483.

Preview cleaned records, engineered inputs and excluded leakage fields.

```
01 | # Cell 45 - Preview cleaned dataset
02 | cleaned_df.head().to_string(index=False)
```

Notebook cell 48

Original file: new.ipynb, JSON source block lines 1510?1517.

Separate 33-column X from the is_canceled target y.

```
01 | # Cell 47 - Define the feature matrix and target
02 | X = cleaned_df.drop(columns=['is_canceled'])
03 | y = cleaned_df['is_canceled']
04 |
05 | print('X shape:', X.shape)
06 | print('y shape:', y.shape)
```

Notebook cell 49

Original file: new.ipynb, JSON source block lines 1536?1549.

Create the only 80/20 split with stratification and seed 42.

```
01 | # Cell 48 - Train-test split with stratification
02 | X_train, X_test, y_train, y_test = train_test_split(
03 |     X, y,
04 |     test_size=0.2,
05 |     random_state=42,
06 |     stratify=y
07 | )
08 |
09 | print('X_train:', X_train.shape)
10 | print('X_test:', X_test.shape)
11 | print('y_train counts:', y_train.value_counts().to_dict())
12 | print('y_test counts:', y_test.value_counts().to_dict())
```

Notebook cell 50

Original file: new.ipynb, JSON source block lines 1566?1573.

Select preliminary numeric/object category groups; string dtype support differs at Stage 6.

```
01 | # Cell 49 - Identify numeric and categorical columns
02 | numeric_cols = X.select_dtypes(include=['number']).columns.tolist()
03 | categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
04 |
05 | print('Numeric columns:', numeric_cols)
06 | print('Categorical columns:', categorical_cols)
```

Notebook cell 51

Original file: new.ipynb, JSON source block lines 1590?1598.

Define numeric template: median imputation plus standard scaling.

```
01 | # Cell 50 - Build a preprocessing pipeline for numeric features
02 | numeric_transformer = Pipeline([
03 |     ('imputer', SimpleImputer(strategy='median')),
04 |     ('scaler', StandardScaler())
05 | ])
06 |
07 | print(numeric_transformer)
```

Notebook cell 52

Original file: new.ipynb, JSON source block lines 1615?1623.

Define categorical template: most-frequent imputation plus one-hot encoding.

```
01 | # Cell 51 - Build preprocessing pipeline for categorical features
02 | categorical_transformer = Pipeline([
03 |     ('imputer', SimpleImputer(strategy='most_frequent')),
04 |     ('onehot', OneHotEncoder(handle_unknown='ignore'))
05 | ])
06 |
07 | print(categorical_transformer)
```

Notebook cell 53

Original file: new.ipynb, JSON source block lines 1663?1671.

Combine numeric/category templates using ColumnTransformer.

```
01 | # Cell 52 - Combine both transformers
02 | preprocessor = ColumnTransformer([
03 |     ('num', numeric_transformer, numeric_cols),
04 |     ('cat', categorical_transformer, categorical_cols)
05 | ])
06 |
07 | print(preprocessor)
```

Notebook cell 54

Original file: new.ipynb, JSON source block lines 1688?1695.

Learn preliminary transforms from training rows only and transform test rows.

```
01 | # Cell 53 - Apply preprocessing to training data
02 | X_train_processed = preprocessor.fit_transform(X_train)
03 | X_test_processed = preprocessor.transform(X_test)
04 |
```

```
05 | print('Processed train shape:', X_train_processed.shape)
06 | print('Processed test shape:', X_test_processed.shape)
```

Notebook cell 55

Original file: new.ipynb, JSON source block lines 1715?1722.

Print stage readiness statements; these do not independently verify correctness.

```
01 | # Cell 54 - Final data readiness check
02 | print('Stage 1 complete: problem defined.')
03 | print('Stage 2 complete: dataset selected and relevant.')
04 | print('Stage 3 complete: EDA performed and insights documented.')
05 | print('Stage 4 complete: preprocessing and feature engineering done.')
06 | print('Stage 5 complete: train/test split ready for model benchmarking.')
```

Notebook cell 57

Original file: new.ipynb, JSON source block lines 1759?1775.

Print hard-coded True checklist entries; distinguish checklist from executable checks.

```
01 | # Cell 56 - Project checklist
02 | checks = {
03 |     'Problem understood': True,
04 |     'Dataset selected': True,
05 |     'Target variable identified': True,
06 |     'Missing values checked': True,
07 |     'Duplicates checked': True,
08 |     'EDA completed': True,
09 |     'Preprocessing applied': True,
10 |     'Leakage reviewed': True,
11 |     'Train/test split done': True
12 | }
13 |
14 | for item, status in checks.items():
15 |     print(f'{item}: {status}')
```

Notebook cell 60

Original file: new.ipynb, JSON source block lines 1824?1854.

Normalize identifiers with a visible loop, clone Stage 5 transformer templates, and define model_preprocessor and stratified folds directly; no custom function.

```
01 | # Shared setup: reuse the split and transformer templates completed in Stage 5.
02 | from sklearn.base import clone
03 | from sklearn.linear_model import LogisticRegression
04 | from sklearn.tree import DecisionTreeClassifier
05 | from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
06 | from sklearn.model_selection import StratifiedKFold, cross_validate
07 |
08 | # Identifiers represent categories, not continuous quantities.
09 | X_model_train = X_train.copy()
10 | for feature in ['agent', 'company']:
11 |     X_model_train[feature] = X_model_train[feature].fillna(0).astype(str)
```

```

12 | numeric_features = X_model_train.select_dtypes(include=[np.number]).columns.tolist()
13 | categorical_features = X_model_train.select_dtypes(
14 |     include=['object', 'string', 'category']
15 | ).columns.tolist()
16 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
17 |
18 | # Reuse the Stage 5 transformer templates without writing custom functions.
19 | model_numeric_transformer = clone(numeric_transformer)
20 | model_categorical_transformer = clone(categorical_transformer)
21 | model_categorical_transformer.set_params(onehot__min_frequency=50)
22 | model_preprocessor = ColumnTransformer([
23 |     ('numeric', model_numeric_transformer, numeric_features),
24 |     ('categorical', model_categorical_transformer, categorical_features)
25 | ])
26 |
27 | print('Training rows:', len(X_model_train), '| Held-out test rows:', len(X_test))
28 | print('Numeric features:', len(numeric_features), '| Categorical features:', len(categorical_features))
29 | print('Validation: 3 stratified training-only folds; no CSV reload or new split.')

```

Notebook cell 62

Original file: new.ipynb, JSON source block lines 1879?1888.

Initialize result containers and seven CV scoring metrics; individual training and result calculations appear explicitly in each model cell.

```

01 | # Shared result containers and metrics; training is written explicitly below.
02 | baseline_rows = []
03 | baseline_pipelines = {}
04 | scoring_metrics = {
05 |     'accuracy': 'accuracy', 'balanced_accuracy': 'balanced_accuracy',
06 |     'precision': 'precision', 'recall': 'recall', 'f1': 'f1',
07 |     'roc_auc': 'roc_auc', 'average_precision': 'average_precision'
08 | }

```

Notebook cell 63

Original file: new.ipynb, JSON source block lines 1976?2006.

Build Logistic Regression pipeline, run cross-validation, calculate metrics, update its result row and display results directly.

```

01 | # Baseline 1 - Logistic Regression: explicit pipeline, training and metrics
02 | logistic_baseline = Pipeline([
03 |     ('preprocess', clone(model_preprocessor)),
04 |     ('model', LogisticRegression(max_iter=1000, random_state=42))
05 | ])
06 | scores = cross_validate(
07 |     logistic_baseline, X_model_train, y_train, cv=cv, scoring=scoring_metrics,
08 |     return_train_score=True, n_jobs=1, error_score='raise'
09 | )
10 | baseline_pipelines['Logistic Regression'] = logistic_baseline
11 | result = {
12 |     'Model': 'Logistic Regression',
13 |     'CV ROC-AUC mean': scores['test_roc_auc'].mean(),
14 |     'CV ROC-AUC std': scores['test_roc_auc'].std(),
15 |     'CV Accuracy': scores['test_accuracy'].mean(),
16 |     'CV Balanced Accuracy': scores['test_balanced_accuracy'].mean(),
17 |     'CV Precision': scores['test_precision'].mean(),

```

```

18 |     'CV Recall': scores['test_recall'].mean(),
19 |     'CV F1': scores['test_f1'].mean(),
20 |     'CV Average Precision': scores['test_average_precision'].mean(),
21 |     'Train-validation ROC-AUC gap': (
22 |         scores['train_roc_auc'].mean() - scores['test_roc_auc'].mean()
23 |     )
24 | }
25 | # Re-running a model cell updates its row rather than duplicating it.
26 | baseline_rows[:] = [row for row in baseline_rows if row['Model'] != 'Logistic Regression']
27 | baseline_rows.append(result)
28 | print('Logistic Regression: mean CV ROC-AUC =', round(result['CV ROC-AUC mean'], 4))
29 | display(pd.DataFrame([result]).round(4))

```

Notebook cell 64

Original file: new.ipynb, JSON source block lines 2094?2124.

Build Decision Tree pipeline, run cross-validation, calculate metrics, update its result row and display results directly.

```

01 | # Baseline 2 - Decision Tree: explicit pipeline, training and metrics
02 | decision_tree_baseline = Pipeline([
03 |     ('preprocess', clone(model_preprocessor)),
04 |     ('model', DecisionTreeClassifier(max_depth=20, min_samples_leaf=5, random_state=42))
05 | ])
06 | scores = cross_validate(
07 |     decision_tree_baseline, X_model_train, y_train, cv=cv, scoring=scoring_metrics,
08 |     return_train_score=True, n_jobs=1, error_score='raise'
09 | )
10 | baseline_pipelines['Decision Tree'] = decision_tree_baseline
11 | result = {
12 |     'Model': 'Decision Tree',
13 |     'CV ROC-AUC mean': scores['test_roc_auc'].mean(),
14 |     'CV ROC-AUC std': scores['test_roc_auc'].std(),
15 |     'CV Accuracy': scores['test_accuracy'].mean(),
16 |     'CV Balanced Accuracy': scores['test_balanced_accuracy'].mean(),
17 |     'CV Precision': scores['test_precision'].mean(),
18 |     'CV Recall': scores['test_recall'].mean(),
19 |     'CV F1': scores['test_f1'].mean(),
20 |     'CV Average Precision': scores['test_average_precision'].mean(),
21 |     'Train-validation ROC-AUC gap': (
22 |         scores['train_roc_auc'].mean() - scores['test_roc_auc'].mean()
23 |     )
24 | }
25 | # Re-running a model cell updates its row rather than duplicating it.
26 | baseline_rows[:] = [row for row in baseline_rows if row['Model'] != 'Decision Tree']
27 | baseline_rows.append(result)
28 | print('Decision Tree: mean CV ROC-AUC =', round(result['CV ROC-AUC mean'], 4))
29 | display(pd.DataFrame([result]).round(4))

```

Notebook cell 65

Original file: new.ipynb, JSON source block lines 2212?2242.

Build Random Forest pipeline, run cross-validation, calculate metrics, update its result row and display results directly.

```

01 | # Baseline 3 - Random Forest: explicit pipeline, training and metrics
02 | random_forest_baseline = Pipeline([

```

```

03 |     ('preprocess', clone(model_preprocessor)),
04 |     ('model', RandomForestClassifier(n_estimators=120, min_samples_leaf=2, n_jobs=-1, random_state=42))
05 | ])
06 | scores = cross_validate(
07 |     random_forest_baseline, X_model_train, y_train, cv=cv, scoring=scoring_metrics,
08 |     return_train_score=True, n_jobs=1, error_score='raise'
09 | )
10 | baseline_pipelines['Random Forest'] = random_forest_baseline
11 | result = {
12 |     'Model': 'Random Forest',
13 |     'CV ROC-AUC mean': scores['test_roc_auc'].mean(),
14 |     'CV ROC-AUC std': scores['test_roc_auc'].std(),
15 |     'CV Accuracy': scores['test_accuracy'].mean(),
16 |     'CV Balanced Accuracy': scores['test_balanced_accuracy'].mean(),
17 |     'CV Precision': scores['test_precision'].mean(),
18 |     'CV Recall': scores['test_recall'].mean(),
19 |     'CV F1': scores['test_f1'].mean(),
20 |     'CV Average Precision': scores['test_average_precision'].mean(),
21 |     'Train-validation ROC-AUC gap': (
22 |         scores['train_roc_auc'].mean() - scores['test_roc_auc'].mean()
23 |     )
24 | }
25 | # Re-running a model cell updates its row rather than duplicating it.
26 | baseline_rows[:] = [row for row in baseline_rows if row['Model'] != 'Random Forest']
27 | baseline_rows.append(result)
28 | print('Random Forest: mean CV ROC-AUC =', round(result['CV ROC-AUC mean'], 4))
29 | display(pd.DataFrame([result]).round(4))

```

Notebook cell 66

Original file: new.ipynb, JSON source block lines 2330?2360.

Build Extra Trees pipeline, run cross-validation, calculate metrics, update its result row and display results directly.

```

01 | # Baseline 4 - Extra Trees: explicit pipeline, training and metrics
02 | extra_trees_baseline = Pipeline([
03 |     ('preprocess', clone(model_preprocessor)),
04 |     ('model', ExtraTreesClassifier(n_estimators=120, min_samples_leaf=2, n_jobs=-1, random_state=42))
05 | ])
06 | scores = cross_validate(
07 |     extra_trees_baseline, X_model_train, y_train, cv=cv, scoring=scoring_metrics,
08 |     return_train_score=True, n_jobs=1, error_score='raise'
09 | )
10 | baseline_pipelines['Extra Trees'] = extra_trees_baseline
11 | result = {
12 |     'Model': 'Extra Trees',
13 |     'CV ROC-AUC mean': scores['test_roc_auc'].mean(),
14 |     'CV ROC-AUC std': scores['test_roc_auc'].std(),
15 |     'CV Accuracy': scores['test_accuracy'].mean(),
16 |     'CV Balanced Accuracy': scores['test_balanced_accuracy'].mean(),
17 |     'CV Precision': scores['test_precision'].mean(),
18 |     'CV Recall': scores['test_recall'].mean(),
19 |     'CV F1': scores['test_f1'].mean(),
20 |     'CV Average Precision': scores['test_average_precision'].mean(),
21 |     'Train-validation ROC-AUC gap': (
22 |         scores['train_roc_auc'].mean() - scores['test_roc_auc'].mean()
23 |     )
24 | }
25 | # Re-running a model cell updates its row rather than duplicating it.
26 | baseline_rows[:] = [row for row in baseline_rows if row['Model'] != 'Extra Trees']
27 | baseline_rows.append(result)

```

```
28 | print('Extra Trees: mean CV ROC-AUC =', round(result['CV ROC-AUC mean'], 4))
29 | display(pd.DataFrame([result]).round(4))
```

Notebook cell 67

Original file: new.ipynb, JSON source block lines 2489?2495.

Sort and display baseline CV results by mean ROC-AUC.

```
01 | # Compare all baseline metrics in one table
02 | baseline_results = pd.DataFrame(baseline_rows).sort_values(
03 |     'CV ROC-AUC mean', ascending=False
04 | ).reset_index(drop=True)
05 | display(baseline_results.round(4))
```

Notebook cell 68

Original file: new.ipynb, JSON source block lines 2521?2537.

Plot CV AUC with fold standard deviations and grouped accuracy/precision/recall/F1 bars.

```
01 | # Baseline comparison: ranking performance and threshold-based metrics.
02 | fig, axes = plt.subplots(1, 2, figsize=(16, 5))
03 | axes[0].bar(
04 |     baseline_results['Model'], baseline_results['CV ROC-AUC mean'],
05 |     yerr=baseline_results['CV ROC-AUC std'], capsize=5, color='steelblue'
06 | )
07 | axes[0].set(ylim=(0, 1), ylabel='Mean 3-fold CV ROC-AUC',
08 |             title='Baseline ROC-AUC (error bars: fold standard deviation)')
09 | baseline_results.set_index('Model')[
10 |     ['CV Accuracy', 'CV Precision', 'CV Recall', 'CV F1']
11 | ].plot.bar(ax=axes[1], ylim=(0, 1), rot=15)
12 | axes[1].set(title='Baseline Metrics', ylabel='Mean validation score')
13 | axes[0].tick_params(axis='x', rotation=15)
14 | plt.tight_layout()
15 | plt.show()
```

Notebook cell 70

Original file: new.ipynb, JSON source block lines 2571?2598.

Write the forest tuning pipeline directly, then evaluate four sampled parameter combinations and refit the best.

```
01 | # Stage 7 - Randomized hyperparameter search for Random Forest
02 | from sklearn.model_selection import RandomizedSearchCV
03 |
04 | rf_search = RandomizedSearchCV(
05 |     estimator=Pipeline([
06 |         ('preprocess', clone(model_preprocessor)),
07 |         ('model', RandomForestClassifier(random_state=42, n_jobs=-1))
08 |     ]),
09 |     param_distributions={
10 |         'model__n_estimators': [100, 160],
11 |         'model__max_depth': [None, 20],
12 |         'model__min_samples_leaf': [2, 5, 10],
13 |         'model__max_features': ['sqrt', 0.7]
```

```

14 |     },
15 |     n_iter=4,
16 |     scoring='roc_auc',
17 |     cv=cv,
18 |     n_jobs=1,
19 |     random_state=42,
20 |     refit=True,
21 |     error_score='raise'
22 | )
23 | rf_search.fit(X_model_train, y_train)
24 |
25 | print('Best Random Forest parameters:', rf_search.best_params_)
26 | print('Best training CV ROC-AUC:', round(rf_search.best_score_, 4))

```

Notebook cell 71

Original file: new.ipynb, JSON source block lines 2622?2645.

Write the Logistic Regression tuning pipeline directly, then evaluate six C/weight combinations and refit the best.

```

01 | # Stage 7 - Grid search for Logistic Regression
02 | from sklearn.model_selection import GridSearchCV
03 |
04 | lr_search = GridSearchCV(
05 |     estimator=Pipeline([
06 |         ('preprocess', clone(model_preprocessor)),
07 |         ('model', LogisticRegression(max_iter=1000, random_state=42))
08 |     ]),
09 |     param_grid={
10 |         'model__C': [0.3, 1.0, 3.0],
11 |         'model__class_weight': [None, 'balanced']
12 |     },
13 |     scoring='roc_auc',
14 |     cv=cv,
15 |     n_jobs=1,
16 |     refit=True,
17 |     error_score='raise'
18 | )
19 | lr_search.fit(X_model_train, y_train)
20 |
21 | print('Best Logistic Regression parameters:', lr_search.best_params_)
22 | print('Best training CV ROC-AUC:', round(lr_search.best_score_, 4))

```

Notebook cell 72

Original file: new.ipynb, JSON source block lines 2741?2759.

Rank four baseline and two tuned candidates using training CV AUC.

```

01 | # Compare baseline and tuned candidates using training-only CV ROC-AUC
02 | comparison_rows = [
03 |     {
04 |         'Model': f'Baseline {row["Model"]}',
05 |         'CV ROC-AUC': row['CV ROC-AUC mean'],
06 |         'Type': 'Baseline'
07 |     }
08 |     for _, row in baseline_results.iterrows()
09 | ]
10 | comparison_rows.extend([

```

```

11 |     {'Model': 'Tuned Random Forest', 'CV ROC-AUC': rf_search.best_score_, 'Type': 'Tuned'},
12 |     {'Model': 'Tuned Logistic Regression', 'CV ROC-AUC': lr_search.best_score_, 'Type': 'Tuned'}
13 | ])
14 | model_comparison = pd.DataFrame(comparison_rows).sort_values(
15 |     'CV ROC-AUC', ascending=False
16 | ).reset_index(drop=True)
17 | display(model_comparison.round(4))

```

Notebook cell 73

Original file: new.ipynb, JSON source block lines 2784?2801.

Clone/refit the top-ranked candidate on all training rows before testing.

```

01 | # Select the highest-CV-ROC-AUC candidate and refit it on all training rows
02 | candidate_pipelines = {
03 |     'Baseline Logistic Regression': baseline_pipelines['Logistic Regression'],
04 |     'Baseline Decision Tree': baseline_pipelines['Decision Tree'],
05 |     'Baseline Random Forest': baseline_pipelines['Random Forest'],
06 |     'Baseline Extra Trees': baseline_pipelines['Extra Trees'],
07 |     'Tuned Random Forest': rf_search.best_estimator_,
08 |     'Tuned Logistic Regression': lr_search.best_estimator_
09 | }
10 | selected_name = model_comparison.loc[0, 'Model']
11 | selected_cv_auc = model_comparison.loc[0, 'CV ROC-AUC']
12 | final_model = clone(candidate_pipelines[selected_name]).fit(X_model_train, y_train)
13 |
14 | print('Selected model:', selected_name)
15 | print('Training CV ROC-AUC:', round(selected_cv_auc, 4))
16 | print('Final model refit on all Stage 5 training rows; test set remains unused.')

```

Notebook cell 74

Original file: new.ipynb, JSON source block lines 2873?2899.

Write the ablated ColumnTransformer directly and evaluate without engineered features using fixed selected settings.

```

01 | # Feature ablation reuses the same preprocessing templates and CV folds.
02 | engineered_features = [
03 |     'total_nights', 'is_long_stay', 'is_weekend_stay', 'is_high_lead_time'
04 | ]
05 | X_train_without_engineering = X_model_train.drop(columns=engineered_features)
06 | ablated_numeric_features = [
07 |     column for column in numeric_features if column not in engineered_features
08 | ]
09 | ablated_pipeline = Pipeline([
10 |     ('preprocess', ColumnTransformer([
11 |         ('numeric', clone(model_numeric_transformer), ablated_numeric_features),
12 |         ('categorical', clone(model_categorical_transformer), categorical_features)
13 |     ])),
14 |     ('model', clone(final_model.named_steps['model']))
15 | ])
16 | ablation_scores = cross_validate(
17 |     ablated_pipeline, X_train_without_engineering, y_train,
18 |     cv=cv, scoring='roc_auc', n_jobs=1, error_score='raise'
19 | )
20 | ablated_cv_auc = ablation_scores['test_score'].mean()
21 | display(pd.DataFrame({

```

```

22 |     'Feature set': ['With engineered features', 'Without engineered features'],
23 |     'CV ROC-AUC': [selected_cv_auc, ablated_cv_auc]
24 | }).round(4))
25 | print('Engineering AUC difference:', round(selected_cv_auc - ablated_cv_auc, 4))

```

Notebook cell 75

Original file: new.ipynb, JSON source block lines 3077?3126.

Use built-in pandas fillna/astype operations in the saved pipeline; compute nine held-out metrics and exact confusion counts.

```

01 | # Final held-out evaluation: only after selecting the best model with training CV.
02 | from sklearn.metrics import (
03 |     accuracy_score, balanced_accuracy_score, precision_score, recall_score,
04 |     f1_score, roc_auc_score, average_precision_score, matthews_corrcoef,
05 |     confusion_matrix, classification_report
06 | )
07 | from sklearn.preprocessing import FunctionTransformer
08 |
09 | deployment_model = Pipeline([
10 |     ('normalize_identifiers', Pipeline([
11 |         ('fill_missing_ids', FunctionTransformer(
12 |             pd.DataFrame.fillna, kw_args={'value': {'agent': 0, 'company': 0}}, validate=False
13 |         )),
14 |         ('string_ids', FunctionTransformer(
15 |             pd.DataFrame.astype, kw_args={'dtype': {'agent': str, 'company': str}}, validate=False
16 |         ))
17 |     ])),
18 |     ('classifier', final_model)
19 | ])
20 | y_test_pred = deployment_model.predict(X_test)
21 | y_test_probability = deployment_model.predict_proba(X_test)[:, 1]
22 | tn, fp, fn, tp = confusion_matrix(y_test, y_test_pred, labels=[0, 1]).ravel()
23 | test_metrics = pd.DataFrame({
24 |     'Metric': ['Accuracy', 'Balanced Accuracy', 'Precision', 'Recall', 'F1',
25 |               'ROC-AUC', 'Average Precision', 'Specificity', 'MCC'],
26 |     'Score': [
27 |         accuracy_score(y_test, y_test_pred),
28 |         balanced_accuracy_score(y_test, y_test_pred),
29 |         precision_score(y_test, y_test_pred, zero_division=0),
30 |         recall_score(y_test, y_test_pred, zero_division=0),
31 |         f1_score(y_test, y_test_pred, zero_division=0),
32 |         roc_auc_score(y_test, y_test_probability),
33 |         average_precision_score(y_test, y_test_probability),
34 |         tn / (tn + fp) if tn + fp else 0.0,
35 |         matthews_corrcoef(y_test, y_test_pred)
36 |     ]
37 | })
38 | print('Selected model:', selected_name)
39 | display(test_metrics.round(4))
40 | print(classification_report(
41 |     y_test, y_test_pred, labels=[0, 1],
42 |     target_names=['Not canceled', 'Canceled'], zero_division=0
43 | ))
44 | display(pd.DataFrame(
45 |     [[tn, fp], [fn, tp]],
46 |     index=['Actual not canceled', 'Actual canceled'],
47 |     columns=['Predicted not canceled', 'Predicted canceled']
48 | ))

```

Notebook cell 76

Original file: new.ipynb, JSON source block lines 3152?3176.

Plot held-out confusion matrix, ROC curve and precision-recall curve.

```
01 | # Visual evaluation: confusion matrix, ROC, and precision-recall curves.
02 | from sklearn.metrics import ConfusionMatrixDisplay, roc_curve, precision_recall_curve
03 |
04 | fig, axes = plt.subplots(1, 3, figsize=(18, 5))
05 | ConfusionMatrixDisplay.from_predictions(
06 |     y_test, y_test_pred, labels=[0, 1],
07 |     display_labels=['Not canceled', 'Canceled'],
08 |     cmap='Blues', ax=axes[0], colorbar=False
09 | )
10 | axes[0].set_title('Held-out Confusion Matrix')
11 | fpr, tpr, _ = roc_curve(y_test, y_test_probability)
12 | axes[1].plot(fpr, tpr, label=f'ROC-AUC = {roc_auc_score(y_test, y_test_probability):.3f}')
13 | axes[1].plot([0, 1], [0, 1], '--', color='grey', label='Random ranking')
14 | axes[1].set(xlabel='False Positive Rate', ylabel='True Positive Rate', title='Held-out ROC Curve')
15 | axes[1].legend()
16 | pr_precision, pr_recall, _ = precision_recall_curve(y_test, y_test_probability)
17 | axes[2].plot(pr_recall, pr_precision,
18 |             label=f'Average precision = {average_precision_score(y_test, y_test_probability):.3f}')
19 | axes[2].axhline(y_test.mean(), linestyle='--', color='grey', label='Cancellation prevalence')
20 | axes[2].set(xlabel='Recall', ylabel='Precision', title='Held-out Precision-Recall Curve')
21 | axes[2].legend()
22 | plt.tight_layout()
23 | plt.show()
```

Notebook cell 77

Original file: new.ipynb, JSON source block lines 3200?3229.

Save the fitted pipeline and metadata, reload, and verify sample labels/probabilities; no helper file is needed.

```
01 | # Save the selected fitted pipeline and metadata; verify the reloaded model.
02 | import joblib
03 | from pathlib import Path
04 |
05 | model_artifact_path = Path('best_hotel_cancellation_model.joblib')
06 | model_artifact = {
07 |     'model': deployment_model,
08 |     'selected_model': selected_name,
09 |     'target': 'is_canceled',
10 |     'feature_columns': X_model_train.columns.tolist(),
11 |     'classification_threshold': 0.5,
12 |     'training_cv_roc_auc': float(selected_cv_auc),
13 |     'test_metrics': dict(zip(test_metrics['Metric'], test_metrics['Score'])),
14 |     'input_description': 'Stage 5 cleaned booking features, including engineered columns'
15 | }
16 | joblib.dump(model_artifact, model_artifact_path, compress=3)
17 | loaded_artifact = joblib.load(model_artifact_path)
18 | verification_row = X_test.iloc[[0]].copy()
19 | assert np.array_equal(
20 |     deployment_model.predict(verification_row),
21 |     loaded_artifact['model'].predict(verification_row)
22 | )
23 | assert np.allclose(
24 |     deployment_model.predict_proba(verification_row),
25 |     loaded_artifact['model'].predict_proba(verification_row)
```

```
26 | )
27 | print('Saved and verified:', model_artifact_path.resolve())
28 | print('Saved pipeline uses built-in pandas operations; no custom helper file is required.')
```

Notebook cell 78

Original file: new.ipynb, JSON source block lines 3512?3532.

Use reloaded model for one held-out booking; show all inputs, label, probability, actual outcome and correctness.

```
01 | # Test ONE held-out booking using the saved/reloaded best model.
02 | sample_position = 0
03 | sample_booking = X_test.iloc[[sample_position]].copy()
04 | sample_prediction = int(loader_artifact['model'].predict(sample_booking)[0])
05 | sample_cancellation_probability = float(
06 |     loader_artifact['model'].predict_proba(sample_booking)[0, 1]
07 | )
08 | sample_actual = int(y_test.iloc[sample_position])
09 | print('Booking input (all required features):')
10 | display(sample_booking.T.rename(columns={sample_booking.index[0]: 'Value'}))
11 | sample_result = pd.DataFrame([
12 |     'Booking index': sample_booking.index[0],
13 |     'Actual outcome': 'Canceled' if sample_actual == 1 else 'Not canceled',
14 |     'Predicted outcome': 'Likely to cancel' if sample_prediction == 1 else 'Not likely to cancel',
15 |     'Cancellation probability': round(sample_cancellation_probability, 4),
16 |     'Prediction correct': sample_prediction == sample_actual
17 | ])
18 | display(sample_result)
19 | print(f'Cancellation probability: {sample_cancellation_probability:.2%}')
```